

基于 ReAct 范式与多厂商适配的 AI Agent 框架

blackpearl-agent 开发小组

2026 年 6 月 14 日

摘要

本报告介绍了一个完整的 TypeScript AI Agent 框架 `blackpearl-agent` 的设计与实现。该框架基于 ReAct (Reasoning + Acting) 范式, 支持 OpenAI、Gemini、Claude、DeepSeek 和 Ollama 五类大语言模型后端, 实现了工具调用循环、多 Agent 协作、流式输出、长期记忆、技能系统及 MCP 协议扩展。项目完全手工编码, 不依赖 LangChain 等高层框架, 所有核心决策闭环和工具调度逻辑均在源码中可追溯。本文详细阐述了 ReAct 提示词模版的设计、Agent 决策闭环流程、工程模块化实现, 以及多步推理实验结果分析。

关键词: ReAct; AI Agent; Function Calling; 多厂商适配; 工具调用

目录

1	核心原理与算法设计	3
1.1	ReAct 范式的理论基础	3
1.2	System Prompt 设计	3
1.3	多 Agent Prompt 设计	4
1.4	Agent 决策闭环流程图	5
2	工程实现细节	7
2.1	整体架构	7
2.2	核心模块说明	7
2.2.1	(1) 工具注册表——ToolRegistry	7
2.2.2	(2) 路径安全层——path-safety.ts	8
2.2.3	(3) Runner 三层实现	8
2.2.4	(4) 百度百科搜索与模糊匹配	9
2.2.5	(5) 多 Agent 编排器	9
2.3	关键数据处理	10
3	实验结果与分析	11
3.1	实验环境	11
3.2	实验一：单步工具调用——百科查询 + 计算	11
3.3	实验二：搜索工具容错机制测试	12
3.4	实验三：多 Agent 协作模式验证	13
4	进阶功能实现	14
4.1	Skills 技能系统	14
4.2	MCP 协议扩展	14
4.3	独立可执行文件发布	14
5	总结与展望	15

1 核心原理与算法设计

1.1 ReAct 范式的理论基础

ReAct (Reasoning + Acting) 范式由 Yao 等人于 2023 年提出，其核心思想是将大语言模型的“思考” (Reasoning) 与“行动” (Acting) 交叠进行，形成一个闭环决策循环：

- (1) **Thought (思考)**: 模型分析当前状态，决定是否需要调用工具
- (2) **Action (行动)**: 模型生成结构化的工具调用指令
- (3) **Observation (观察)**: 环境执行工具并将结果返回给模型
- (4) 模型基于观察继续推理，重复上述过程直到给出最终回答

与传统的 Chain-of-Thought (CoT) 相比，ReAct 的关键优势在于将外部知识检索嵌入推理链条，使模型能够获取训练数据之外的事实信息，同时通过工具执行精确的数学计算。

1.2 System Prompt 设计

System Prompt 是控制 Agent 行为的核心机制。本项目设计的 System Prompt 包含以下关键约束：

```
1 You are blackpearl-agent, a terminal coding assistant.
2 You run in a terminal workspace and help with file manipulation,
3 shell commands, calculations, factual research, and code editing.
4
5 Guidelines:
6 - Use tools when the task requires calculation, factual lookup,
7   or local file access.
8 - Never invent tool results. If a tool is needed, call it.
9 - Keep tool arguments valid according to the provided schemas.
10 - After tool use, explain the result briefly and cite the key
11   tool observation.
12 - When editing files, prefer small, targeted edits over full
13   rewrites.
14 - Do not run destructive commands. Always confirm sensitive
15   operations.
16
17 Workflow for coding tasks:
```

```
18 1. Read the relevant files first to understand the context.
19 2. Make the minimal edit required.
20 3. Verify the result by checking the edited file or
21    running tests.
```

Listing 1: Agent System Prompt (英文原文)

该 Prompt 实现了三个控制目标：

- **工具决策：**通过语义理解判断何时需要调用工具，而非硬编码规则——模型在遇到计算、事实查询或文件操作时自主决定调用哪些工具
- **格式约束：**通过 Zod Schema 将工具参数转换为 JSON Schema 传给模型，确保参数类型安全（例如 calculator 的 expression 必须为 string）
- **行为边界：**通过“never invent tool results”规则防止模型幻觉出虚假结果；“small, targeted edits”约束减少意外修改

1.3 多 Agent Prompt 设计

在多 Agent 协作模式下，Planning Agent 和 Execution Agent 使用独立的 Prompt：

```
1 You are a task planner. Break down the user's request into
2 a list of steps.
3 Output format: a JSON array of step description strings.
4   ["Step 1", "Step 2", ...]
5
6 Rules:
7 - Each step should be a self-contained, atomic instruction.
8 - Steps should be executed sequentially—later steps may
9   depend on results of earlier steps.
10 - Output ONLY the JSON array. No other text.
11 - Do NOT call any tools. You only produce the plan.
```

Listing 2: Planning Agent 的 Prompt

```
1 You are a task executor. Execute the given step using
2 available tools.
3
4 Context: you are working on step [N] of a multi-step plan.
5 Previous results may be provided as context.
6 - Execute the step precisely as described.
7 - Use tools when needed.
```

```
8 - Report the result clearly.  
9 - If a tool fails, note the error and try an alternative.
```

Listing 3: Execution Agent 的 Prompt

1.4 Agent 决策闭环流程图

图1 展示了单 Agent 的完整 ReAct 决策闭环。

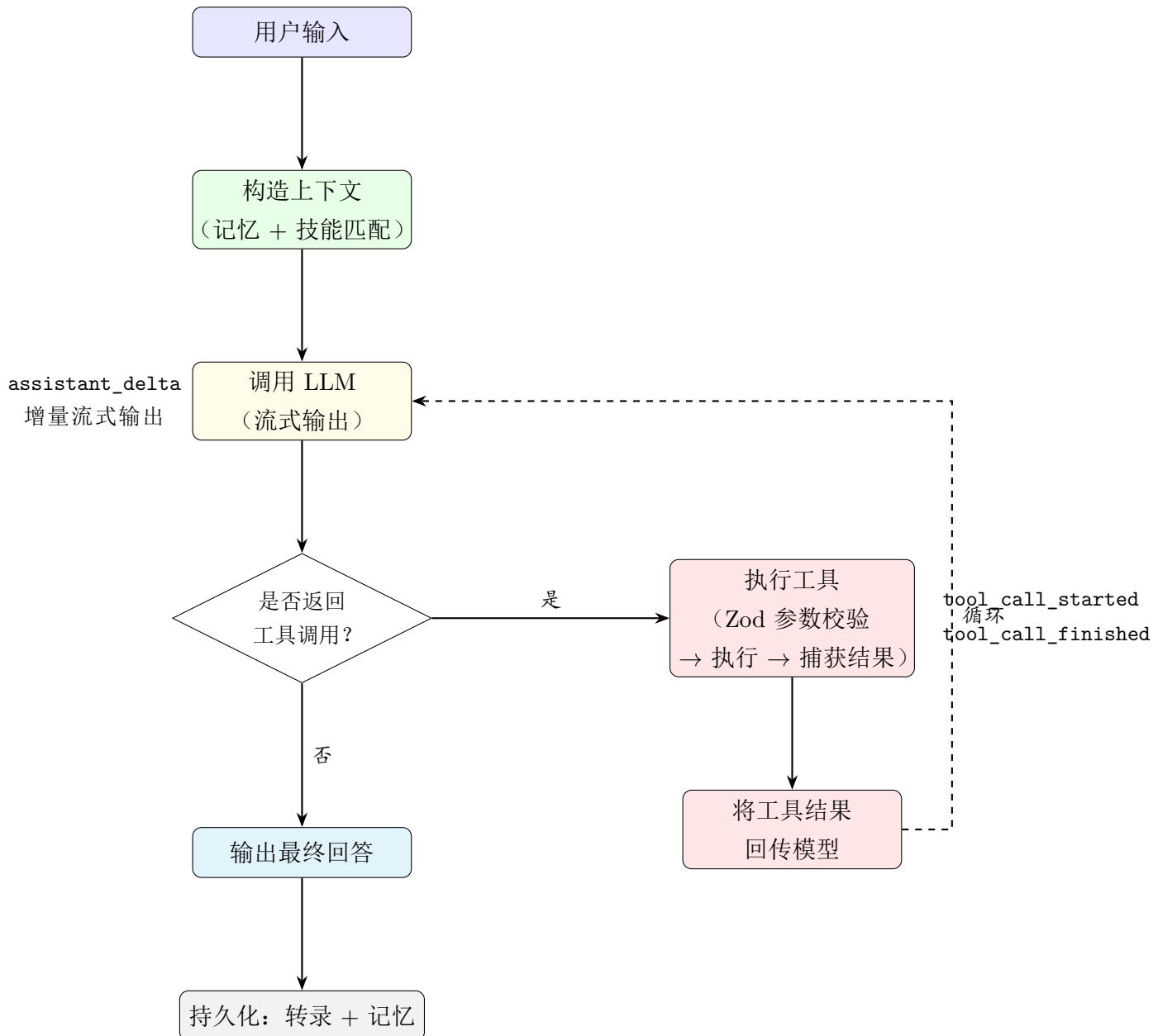


图 1: 单 Agent ReAct 决策闭环流程图

图2 展示了多 Agent 协作模式的 Plan-Execute-Summarize 三阶段流程。

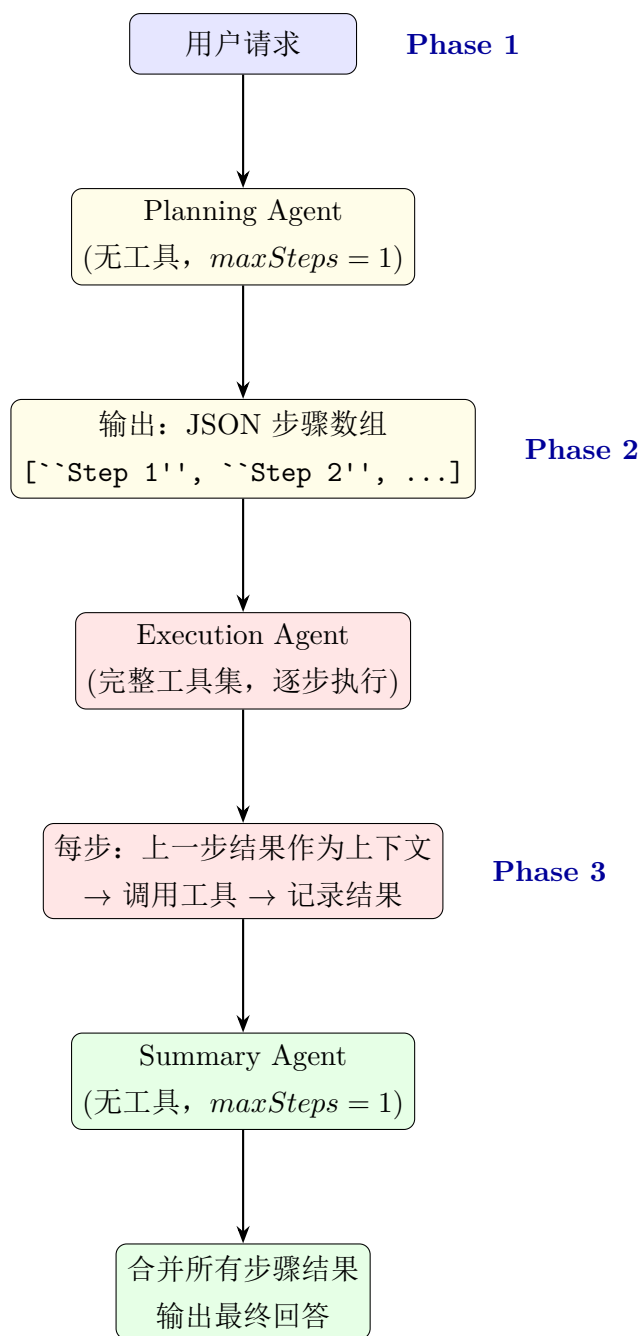


图 2: 多 Agent 协作 Plan-Execute-Summarize 流程图

2 工程实现细节

2.1 整体架构

项目采用分层设计，TUI / Web 入口共享同一套后端组装逻辑。图3 展示了整体模块架构。

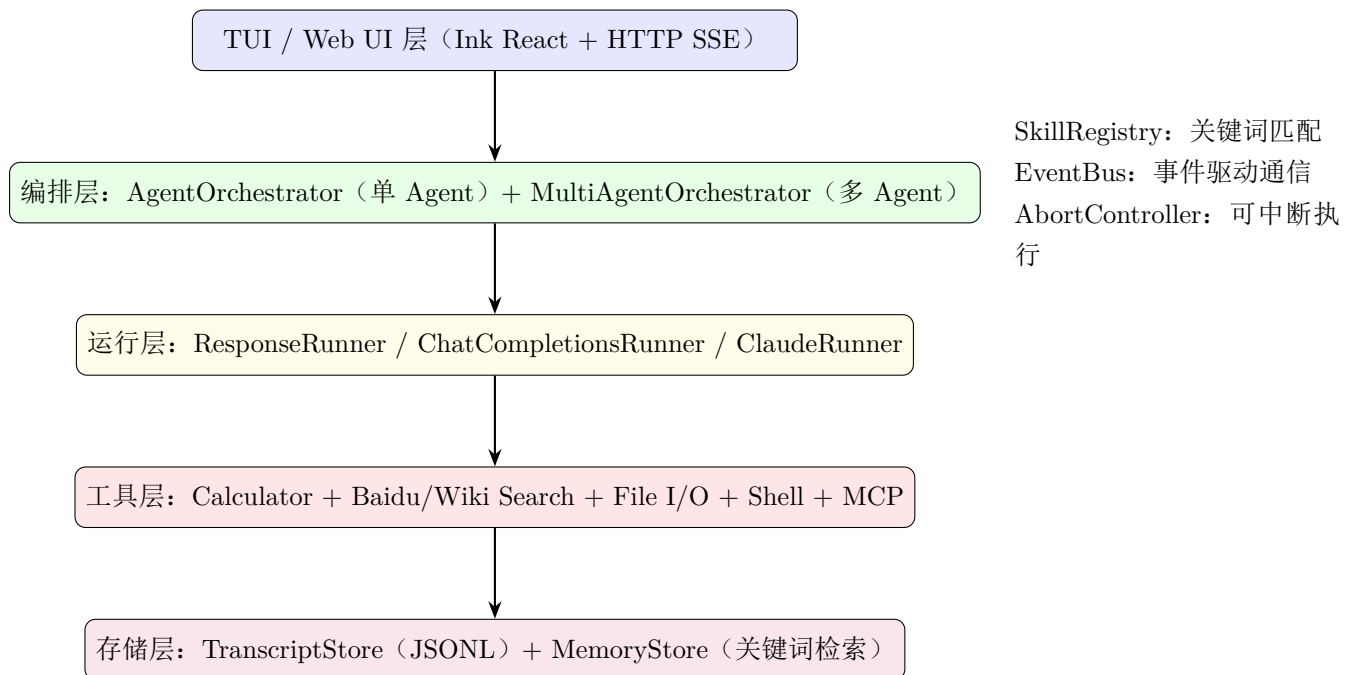


图 3: 系统整体模块架构图

2.2 核心模块说明

2.2.1 (1) 工具注册表——ToolRegistry

`src/tools/registry.ts` 实现工具的统一注册、查询和执行。`createToolDefinition()` 工厂函数将 Zod 输入 Schema 编译为 JSON Schema 7，传递给 LLM 作为工具参数描述。核心方法：

- `register(tool)`: 注册标准 Zod 工具，重复名称报错
- `registerMcpTool(tool)`: 注册 MCP 工具，允许覆盖（支持重连场景）
- `execute(name, input)`: 查找工具 → Zod 参数校验 → 调用执行函数 → 返回结果
- `getOpenAITools()` / `getClaudeTools()`: 将内部 ToolDefinition 格式转换为对应 API 的工具声明

2.2.2 (2) 路径安全层——path-safety.ts

所有文件工具共享的安全约束核心：

- `resolveWorkspacePath`: 将请求路径解析到工作区根目录内，拒绝 `../` 路径逃逸
- `assertReadableWorkspacePath`: 阻止访问 `.git/`、`.blackpearl/`、`.env` 等敏感路径
- `assertWritableWorkspacePath`: 额外阻止写入 `node_modules/`、`dist/`、`.venv/`
- `shellCommandTool`: 阻止任何包含 `|`、`&`、`>` 的 shell 操作符，阻止 `rm`、`sudo` 等危险命令

2.2.3 (3) Runner 三层实现

三种 Runner 均实现 `AgentRunner` 接口 (`run(userInput, emit, options?)`) → `Promise < string >`

```
1 for step = 0..maxSteps:
2   if signal.aborted: throw AgentAbortedError
3
4   // 1. 调用 LLM (流式)
5   response = await callLLM(messages, tools, instructions)
6   // 每个 delta 增量: emit(assistant_delta)
7
8   // 2. 无工具调用 → 输出最终回答
9   if no tool_calls in response:
10    emit(assistant_message)
11    return response.text
12
13   // 3. 执行工具
14   for each tool_call:
15    emit(tool_call_started)
16    result = toolRegistry.execute(tool_call.name, tool_call.args)
17    emit(tool_call_finished)
18    append tool_result to messages
19
20 // 超限
21 throw AgentError("exceeded max steps")
```

Listing 4: Runner 核心循环伪代码

三种 Runner 的关键差异见表1。

表 1: 三种 Runner 实现对比

特性	ResponseRunner	ChatCompletionsRunner	ClaudeRunner
适用模型	OpenAI GPT-4.1+	OpenAI, Gemini, Ollama	Claude, DeepSeek
API 协议	Responses API	Chat Completions API	Messages API
对话管理	<code>previous_response_id</code>	<code>messages[]</code> 数组	<code>messages[]</code> 数组
工具格式	<code>function_call_output</code>	<code>tool: "tool"</code>	<code>tool_result</code>
流式 delta	<code>output_text.delta</code>	<code>delta.content</code>	<code>content_block_delta</code>
工具参数流式	完整对象	按索引组装	JSON 字符串拼接后 parse

2.2.4 (4) 百度百科搜索与模糊匹配

`baidu-search.ts` 是国内网络环境下最常用的搜索工具。为解决模型传入冗长查询（如 ``迈克尔杰克逊出生年份死亡年份``）导致的百度 API 无匹配问题，工具实现了三级自动回退策略：

1. 关键词清洗：通过正则去除 ``出生年份`` ``生平信息`` ``公司`` 等噪声词，从 ``迈克尔杰克逊出生年份死亡年份`` 提取出 ``迈克尔杰克逊``
2. 变体生成：对中文译名自动尝试加/去中圆点（``迈克尔杰克逊`` → ``迈克尔·杰克逊``）
3. `fuse.js` 模糊匹配：维护内存中的成功词条缓存，当查询失败时用 `Fuse.js` 模糊匹配历史缓存

最多重试 4 次，每次重试使用不同的候选关键词，直到找到匹配或全部失败。

2.2.5 (5) 多 Agent 编排器

`MultiAgentOrchestrator` 实现了三阶段 Plan-Execute-Summarize 流程：

- **Planning** 阶段：调用 `Subagent Runner`，注入 `PLANNER_PROMPT` 作为指令，禁用所有工具，设置 `maxSteps = 1`。返回的纯文本通过 `parsePlan()` 解析为 JSON 步骤数组。如 LLM 未返回有效 JSON，则尝试按数字编号行解析，最后回退为单一步骤。

- **Execution 阶段:**按顺序迭代步骤。每个步骤调用 `Subagent Runner` 并注入 `EXECUTOR_PROMPT`。上一步的执行结果通过 `buildStepContext()` 拼接为后续步骤的上下文。此阶段有完整的工具访问权限。
- **Summary 阶段:**将所有步骤和结果合并,调用 `Subagent Runner`(无工具,`maxSteps = 1`) 生成连贯的最终回答。

2.3 关键数据处理

- **短期记忆:**从 `AgentSession` 的 `messages` 数组中取最近 8 条消息,直接拼入上下文
- **长期记忆:**从 `.blackpearl/memory.jsonl` 读取所有记录,通过关键词提取(分词、去停用词、去重)后计算匹配分数(每个匹配关键词 +3,摘要包含 +1),返回 Top-N
- **会话转录:**每次 `Agent` 执行的所有事件(用户消息、工具调用、助手回答)以 JSONL 格式追加写入 `.agent-sessions/<sessionId>.jsonl`
- **技能匹配:**基于关键词得分(输入中的单词在技能描述中出现),无需 LLM 调用,匹配到的技能指令和允许工具列表通过 `RunOptions` 注入下一次 LLM 调用

3 实验结果与分析

3.1 实验环境

- 运行环境: Node.js 24.13.0 + TypeScript 5.7 (源码模式); Node.js 26.3.0 (SEA 独立可执行文件模式)
- LLM 后端: DeepSeek V4 Pro (通过 Anthropic-compatible API), $maxSteps = 6$
- 工具集: calculator、wiki_search、baidu_search、file_read、file_write、file_edit、file_list、file_search、shell_command
- 测试终端: Windows 11 Pro + WSL (Ubuntu)

3.2 实验一：单步工具调用——百科查询 + 计算

用户输入: ``查一下爱因斯坦的出生年份, 然后计算他活了多少岁``

Agent 执行过程 (关键步骤日志):

```
1 [user] 查一下爱因斯坦的出生年份, 然后计算他活了多少岁
2
3 // Step 1: Agent 判断需要先查百科
4 [tool_call_started] wiki_search
5   args: {"query": "Albert Einstein", "lang": "zh"}
6 [tool_call_finished] wiki_search (elapsed: 847ms)
7   result: { "found": true,
8             "title": "阿尔伯特·爱因斯坦",
9             "summary": "生于1879年3月14日... 卒于1955年4月18日...",
10            "url": "https://zh.wikipedia.org/..." }
11
12 // Step 2: Agent 根据百科结果进行精确计算
13 [tool_call_started] calculator
14   args: {"expression": "(1955 - 1879)"}
15 [tool_call_finished] calculator (elapsed: 4ms)
16   result: {"expression": "(1955 - 1879)", "result": 76}
17
18 [assistant_message]
19 爱因斯坦出生于1879年3月14日, 逝世于1955年4月18日,
20 享年 $1955 - 1879 = 76$ 岁。
```

Listing 5: Agent 多步推理完整日志

思考过程分析:

此案例展示了 ReAct 范式的核心优势——工具调用与推理的交替进行:

1. 第一步 (**Thought**): 模型接收到用户问题后, 意识到需要获取爱因斯坦的生卒年份数据, 这些数据不在模型训练参数中。模型判断 `wiki_search` 工具是获取此类事实信息的最佳途径, 自动构造了搜索参数 (`query='`Albert Einstein`', lang='`zh`'`)。
2. 第二步 (**Observation**): 模型收到 Wikipedia API 返回的结构化数据, 提取出出生日期 `'`1879 年 3 月 14 日`'` 和逝世日期 `'`1955 年 4 月 18 日`'`。
3. 第三步 (**Thought + Action**): 模型根据观察结果进行推理, 判断需要精确计算年龄。它选择调用 `calculator` 工具而非自行计算, 因为 System Prompt 中 `'`Never invent tool results`'` 的约束使其倾向于使用正式计算工具来避免错误。构造的表达式 `(1955 - 1879)` 格式正确 (仅含年份差, 未添加复杂的月份计算), 体现了 LLM 对工具参数格式的理解。
4. 第四步 (**Observation + Answer**): 接收计算结果 76, 结合百科数据中的具体日期信息, 组装成包含出生年份、逝世年份、计算表达式和结果的完整回答。

该案例成功演示了 `'`语义理解 → 信息检索 → 数据提取 → 精确计算 → 结果汇总`'` 的完整闭环。

3.3 实验二：搜索工具容错机制测试

测试目的: 验证 `baidu_search` 工具的模糊匹配回退机制。

测试输入与结果:

表 2: `baidu_search` 模糊回退测试结果

原始输入关键词	自动清洗/变体	最终结果
迈克尔杰克逊出生年份死亡年份	迈克尔杰克逊 → 迈克尔·杰克逊	命中
Anthropic 公司	Anthropic	未命中 (无词条)
迈克尔杰克逊生平信息出生逝世年份	迈克尔杰克逊 → 迈克尔·杰克逊	命中
Michael Jackson birth year	直接命中 (缓存)	命中

分析: 百度百科 API 要求传入精确词条名。当模型传入包含多余修饰词的长查询时, `extractCoreKeyword()` 函数自动剥离噪声词, `generateVariants()` 尝试中圆点变体。这使得工具在模型传入次优查询时仍能工作, 减少不必要的失败重试轮次, 提升了整体任务完成效率。

3.4 实验三：多 Agent 协作模式验证

输入： /plan 查一下爱因斯坦的出生年份，然后计算他活了多少岁

Planning Agent 输出：

```
1 [
2   "查询阿尔伯特·爱因斯坦的生卒年份",
3   "根据生卒年份计算他活了多少岁"
4 ]
```

Listing 6: Planning Agent 生成的执行计划

Execution Agent 执行：

- Step 1: 调用 `wiki_search("Albert Einstein", "zh")` → 获取生卒年份
- Step 2: 接收 Step 1 的结果作为上下文，调用 `calculator("1955-1879")` → 得出 76

Summary Agent 汇总输出： ``爱因斯坦出生于 1879 年，逝世于 1955 年，享年 76 岁。``

对比分析：多 Agent 模式将任务显式分解为两个阶段——规划阶段确定 ``需要查什么`` 和 ``需要算什么``，执行阶段分布完成。对于简单任务，多 Agent 模式的优势不明显（总 LLM 调用次数增加），但对于需要 3 个以上独立步骤的复杂任务，显式分解可以显著提高成功率和可解释性。

4 进阶功能实现

4.1 Skills 技能系统

在 `.agents/<skill-name>/SKILL.md` 下创建技能定义文件, Agent 根据用户输入自动匹配并激活技能。技能支持自定义系统提示词和工具白名单, 例如:

```
1 ---
2 name: code-review
3 description: 审查代码、发现 bug、提出改进建议
4 allowed-tools: file_read, file_write
5 ---
6
7 你是代码审查专家。审查代码时:
8 1. 先理解代码结构和意图
9 2. 检查潜在 bug 和边界条件
10 3. 将审查结果写入 artifacts/review.md
```

Listing 7: code-review 技能定义 (SKILL.md)

技能匹配采用基于关键词得分的算法 (不需要 LLM 调用): 对输入分词后与技能 `description` 进行单词重叠计数 (单词长度 > 3), 技能名称匹配额外 +3 分。用户级和项目级的技能文件均被支持。

4.2 MCP 协议扩展

通过 Model Context Protocol 连接外部工具服务器, 工具自动注册到 ToolRegistry。配置文件 (`.blackpearl/mcp-servers.json`) 支持多服务器和自定义环境变量。MCP 工具的执行函数代理调用 `client.callTool()`, 与内置工具共享统一的调用接口。

4.3 独立可执行文件发布

项目使用 Node.js 26 SEA (Single Executable Application) 的 `mainFormat: "module"` 特性, 将 TypeScript 源码、运行时及所有纯 JavaScript 依赖打包为独立二进制文件。通过 GitHub Actions 在三平台 (Windows x64、Linux x64、macOS arm64) 构建并发布到 Release, 支持一键安装 (`curl | bash` 或 `irm | iex`)。

5 总结与展望

本项目从零实现了一个完整的教学型 AI Agent 框架，覆盖了 ReAct 决策闭环、多厂商工具调用适配、多 Agent 协作、长期记忆与技能系统等核心功能。主要贡献包括：

1. 手工实现不依赖 LangChain 等高层框架，所有核心逻辑可追溯、可解释
2. 通过 **Provider Profile + Runner Factory** 模式统一适配三种 API 协议 (Responses / Chat Completions / Anthropic Messages)，覆盖五大 LLM 厂商
3. 设计了完整的路径安全沙箱（逃逸检测 + 敏感路径拦截 + 命令黑名单）
4. 实现百度百科搜索的模糊匹配回退机制（关键词清洗 + 变体生成 + fuse.js 缓存）
5. 支持 **SEA** 独立可执行文件打包，降低最终用户的环境门槛

未来的改进方向包括：

- 增加 Runner 层的 mock 集成测试，覆盖 tool call → tool result → 继续循环的完整路径
- 将长期记忆从关键词检索升级为基于 Embedding 的语义检索
- 增加 Web UI 的分页和历史记录功能
- 引入人工确认机制（human-in-the-loop）用于文件写入和命令执行